

Fondamenti dell'Informatica

THOMAS TOGNETTI¹

¹www.example.com

For the greater good.

Indice

1	Introduzione e fondamenti	3
1.1	Fondamenti matematici	4
1.1.1	Teoria degli insiemi	4
1.1.2	Successioni matematiche e tuple	7
1.1.3	Funzioni e relazioni	8
1.1.4	Grafi	9
1.2	Stringhe	12
1.2.1	Operazioni sulle stringhe	12
I	Concetti di base	13
2	Rappresentazione dell'informazione	15
2.1	Sistemi di numerazione	15
2.2	Conversioni di base	15
2.3	Aritmetica binaria	15
2.4	Rappresentazione dei numeri interi	15
2.5	Numeri in virgola mobile (IEEE 754)	15
2.6	Rappresentazione di caratteri (ASCII e Unicode)	15
2.7	Rappresentazione di immagini, audio e video	15
3	Algebra di Boole e circuiti logici	17
3.1	Operatori e porte logiche	17
3.2	Espressioni booleane e tabelle di verità	17
3.3	Semplificazione tramite mappe di Karnaugh	17
3.4	Circuiti combinatori	17
3.5	Circuiti sequenziali	17
3.6	Dalle porte logiche alla CPU	17
II	Automi e linguaggi formali	19
4	Automa a stati finiti	21
4.1	Automa a stati finiti deterministico	22

4.2	Automa a stati finiti non deterministico	22
4.3	Rappresentazioni di ASF (Automi a Stati Finiti)	22
4.3.1	Diagramma degli stati	22
4.3.2	Tabella di transizione di stato	23
4.4	Linguaggio di un ASF	23
5	Linguaggi regolari ed espressioni regolari	25
5.1	Le operazioni sui linguaggi formali	26
5.1.1	Unione tra linguaggi	26
5.1.2	Intersezione di linguaggi	26
5.1.3	Concatenazione di linguaggi	26
5.1.4	Complemento di linguaggio	26
5.1.5	Quoziente destro di linguaggi	27
5.1.6	Stella di Kleene	27
5.1.7	Riflesso di linguaggio	27
5.1.8	Mescolamento (Shuffle) tra linguaggi	27
5.2	Espressioni regolari	27
5.3	Linguaggi non regolari	28
5.3.1	Pumping lemma per i linguaggi regolari	28
6	Linguaggi context-free	31
6.1	Introduzione alle grammatiche generative	31
6.2	Grammatiche context-free (CFG)	31
6.3	Alberi di derivazione e ambiguità	31
6.4	Forma normale di Chomsky	31
6.5	Automi a pila (PDA)	31
6.6	Equivalenza tra CFG e PDA	31
6.7	Pumping lemma per i linguaggi context-free	31
7	Macchine di Turing e calcolabilità	33
7.1	Definizione formale della macchina di Turing	33
7.2	Varianti della macchina di Turing	33
7.3	Tesi di Church-Turing	33
7.4	Codifica di macchine di Turing e problemi di decisione	33
7.5	Il problema della fermata (halting problem)	33
7.6	Problemi indecidibili	33
III	Teoria della computabilità	35
8	Funzioni ricorsive e modelli di calcolo	37
8.1	Funzioni primitive ricorsive	37
8.2	Funzioni μ -ricorsive	37
8.3	Equivalenza con le macchine di Turing	37

8.4	La tesi di Church	37
9	Indecidibilità e riducibilità	39
9.1	Insiemi ricorsivi e ricorsivamente enumerabili	39
9.2	Riduzioni many-one e Turing	39
9.3	Teorema di Rice	39
9.4	Gerarchia di Chomsky	39
IV	Teoria della complessità	41
10	Classi di complessità	43
10.1	Misura della complessità temporale e spaziale	43
10.2	Notazione asintotica O-grande, Ω -grande, Θ -grande	43
10.3	Classi P e NP	43
10.4	Problemi NP-completi e riduzioni polinomiali	43
10.5	Il teorema di Cook-Levin	43
10.6	Oltre NP: classi PSPACE, EXP	43
11	Trattabilità e intrattabilità	45
11.1	Algoritmi approssimati	45
11.2	Classi di complessità nello spazio (L, NL, PSPACE)	45
11.3	Gerarchie di complessità e problemi aperti (P vs NP)	45
12	Conclusioni	47

Elenco delle figure

1.1	Diagramma di un insieme	4
1.2	Diagramma dell'unione tra insiemi	5
1.3	Diagramma dell'intersezione tra insiemi	6
1.4	Diagramma della differenza tra insiemi	6
1.5	Diagramma della differenza simmetrica tra insiemi	6
1.6	Diagramma di un piano cartesiano	7
1.8	Diagramma dell'astrazione di una funzione	9
1.9	Diagramma di un grafo non orientato	10
1.10	Diagramma di un sottografo in un grafo non orientato	10
1.11	Diagramma di un grafo orientato	11
1.12	Diagramma di un grafo ad albero	11
4.1	Grafo orientato di un ASF	22

Elenco delle tabelle

1.1	Tavole di verità degli operatori logici AND, OR, XOR e NOT.	9
-----	---	---

Prefazione

Il seguente libro tratterà un corso sui fondamenti dell'informatica fino alla teoria della complessità computazionale, passando per gli automi a stati finiti e per la teoria della computabilità. Si consiglia la lettura di questo libro come di un corso preparatorio al libro *Algoritmi & Strutture Dati 1*, in quanto fornisce le basi per l'analisi della complessità computazionale e, più in generale, le basi sulle quali qualsiasi algoritmo e qualsiasi elaboratori si appoggiano.

1

Introduzione e fondamenti

“Computations are everywhere, once you begin to look at things in a certain way.”

– Rudy Rucker

Oggi la computazione fa parte della vita di tutti quanti ed è tutt'intorno a noi, anche quando non ne siamo consapevoli. Partendo dallo smartphone, fino ad arrivare ai personal computer più potenti, passando per termostati, auto, apparati stereo, frigoriferi, piani cottura ad induzione, porte automatiche, ascensori, orologi da polso, televisori, calcolatrici e così via. I dispositivi che compiono computazioni, detti *elaboratori*, possono essere più o meno potenti, richiedere più o meno energia, occupare più o meno spazio e così via, a seconda delle necessità di elaborazione. Ebbene, in questo libro tenteremo di dare una risposta alla domanda: cos'è un elaboratore?

Questa domanda può essere analizzata secondo vari punti di vista. Potremmo parlare di elaboratore analizzandone la struttura della parte tangibile, fisica (*hardware*) degli elaboratori odierni, oppure analizzare come un elaboratore d'oggi riesca ad effettuare tutte le operazioni e a fornirci tutti i servizi che utilizziamo ogni giorno, ovvero analizzarne le strutture astratte, non tangibili (*software*). Lo scopo di questo libro, tuttavia, non rientra in nessuno di questi. In questo libro l'obiettivo è descrivere con formalismo matematico il funzionamento di un elaboratore, ovvero fornire la base sulla quale tutte le analisi di algoritmi e la classificazione dei problemi di computazione si appoggiano. Comprenderemo, quindi, a livello profondo, quali sono le capacità ed i limiti matematici di un elaboratore. Parleremo allora di *teoria della complessità computazionale*, nella quale riusciremo a comprendere le diverse classi di problemi di computazione e a stabilire quali problemi possono essere categorizzati come complessi e quali come semplici, di *teoria della computabilità*, nella quale affrontiamo la dura realtà della computabilità, la quale vuole che non tutti i problemi siano risolvibili mediante un algoritmo di computazione e di automi, ovvero di com'è possibile rappresentare matematicamente un elaboratore.

1.1 Fondamenti matematici

Prima di affrontare concetti più complessi, tuttavia, è necessario fornire parte della terminologia fondamentale per comprendere alcune definizioni e dimostrazioni fondamentali.

1.1.1 Teoria degli insiemi

Uno dei concetti fondamentali è quello di *insieme*, nella concezione matematica. Per cui, di seguito si fornisce una definizione formale e si analizzano nel dettaglio le operazioni sugli insiemi.

Def. 1.1 *Un insieme è definito come una collezione di elementi o membri, in cui ripetizione ed ordinamento non hanno valore.*

Ipotizzando di avere l'insieme $\mathbb{N}$ di tutti i numeri naturali, otterremmo un diagramma come il seguente.

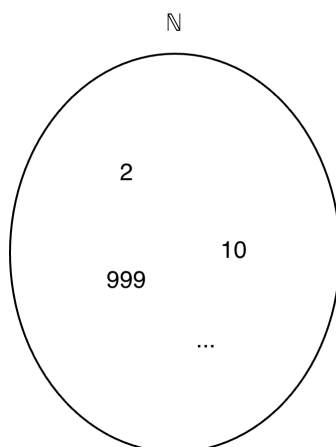


Figura 1.1: Diagramma di un insieme

L'appartenenza di un elemento ad un insieme è rappresentata dalla notazione $e \in I$, che può essere letta come: e è elemento dell'insieme I .

Definizione per elencazione (*Roster*)

Un insieme può essere definito utilizzando la notazione *Roster*, introdotta da Ernst Zermelo (1871-1953), che consiste nell'elencazione degli *elementi* o dei *membri* dell'insieme. Ipotizzando di voler definire l'insieme A dei caratteri che compongono il nome *Thomas*, otteniamo:

$$A = \{T, h, o, m, a, s\}$$

Definizione per proprietà caratteristica

La definizione per elencazione, tuttavia, non è sempre una strada percorribile. Ebbene, in nostro soccorso occorre la definizione per *proprietà caratteristica*, che riesce a definire gli elementi di un insieme grazie ad una definizione logica di una caratteristica distintiva detta *proprietà* che permette di identificarli.

In questo caso, si usa la notazione

$$I = \{x \mid P(x)\},$$

dove I è il nome dell'insieme, x identifica un singolo elemento dell'insieme e $P(x)$ rappresenta la proprietà che un elemento dell'insieme I deve soddisfare per essere tale. Questa definizione può essere letta come *l'insieme I di tutti gli elementi x tali che x soddisfa la proprietà $P(x)$* .

Cardinalità di un insieme

Def. 1.2 La cardinalità $|I|$ o $\#I$ di un insieme rappresenta il numero totale dei suoi elementi.

Un insieme vuoto (cioè che non contiene alcun elemento), indicato dal simbolo $\emptyset$, avrà quindi cardinalità zero, mentre l'insieme dei numeri naturali $\mathbb{N}$ avrà cardinalità ∞ .

Sono detti insiemi *finiti* gli insiemi la cui cardinalità $|I|$ è un numero finito e insiemi *infiniti* tutti quegli insiemi con cardinalità $|I| = \infty$.

Sottoinsiemi

Def. 1.3 L'insieme B è sottoinsieme dell'insieme A se l'insieme A contiene tutti gli elementi che contiene anche l'insieme B .

Secondo questa definizione, tutti gli insiemi sono sottoinsiemi di sé stessi. Si distinguono, poi, i sottoinsiemi propri indicati dalla notazione $B \subset A$, in cui l'insieme A non corrisponde propriamente all'insieme B ma contiene altri elementi $e \notin B$, dai sottoinsiemi impropri, indicati dalla notazione $B \subseteq A$, che indicano insiemi *coincidenti*.

Unione tra insiemi

La prima operazione sugli insiemi che presentiamo è quella di *unione*, definita formalmente nel seguente modo.

Def. 1.4 L'unione di due insiemi A e B è data dall'insieme risultante $A \cup B$ definito per proprietà caratteristica come:

$$A \cup B = \{x \mid x \in A \vee x \in B\}$$

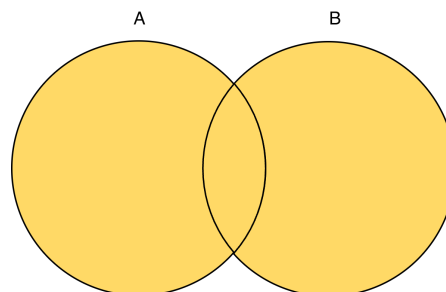


Figura 1.2: Diagramma dell'unione tra insiemi

Data la definizione, possiamo dire che l'unione di due insiemi è un altro insieme composto dagli elementi di entrambi gli insiemi.

Intersezione tra insiemi

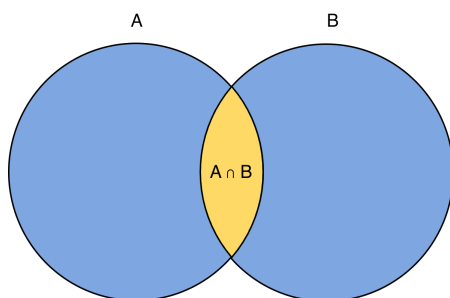


Figura 1.3: Diagramma dell'intersezione tra insiemi

Una seconda operazione sugli insiemi è quella di *intersezione*, la cui definizione formale è:

Def. 1.5 *L'intersezione di due insiemi A e B è data dall'insieme risultante $A \cap B$ definito per proprietà caratteristica come:*

$$A \cap B = \{x \mid x \in A \wedge x \in B\}$$

Possiamo quindi dire che l'intersezione tra due insiemi è un altro insieme dato dai soli elementi in comune di entrambi gli insiemi.

Differenza tra insiemi

Un'altra operazione sugli insiemi è quella della *differenza*, definita come:

Def. 1.6 *La differenza tra due insiemi A e B è data dall'insieme risultante $A \setminus B$ definito per proprietà caratteristica come:*

$$A \setminus B = \{x \mid x \in A \wedge x \notin B\},$$

corrispondente all'insieme degli elementi in un solo insieme, come nel diagramma a destra:

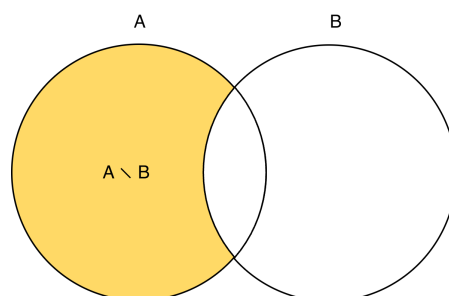


Figura 1.4: Diagramma della differenza tra insiemi

Differenza simmetrica tra insiemi

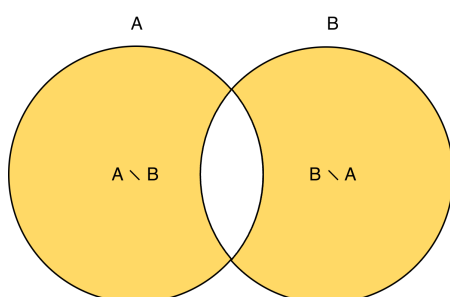


Figura 1.5: Diagramma della differenza simmetrica tra insiemi

La differenza simmetrica tra insiemi è definita come:

Def. 1.7 *La differenza simmetrica tra due insiemi A e B è data dall'insieme risultante $A \Delta B$ definito per proprietà caratteristica come:*

$$A \Delta B = (A \setminus B) \cup (B \setminus A),$$

visualizzabile come il diagramma a sinistra.

Prodotto cartesiano tra insiemi

L'operazione del *prodotto cartesiano* è definita come:

Def. 1.8 Il prodotto cartesiano di due insiemi A e B è dato dall'insieme risultante $A \times B$ definito per proprietà caratteristica come:

$$A \times B = \{(x, y) \mid x \in A \wedge y \in B\}$$

Che può essere letto come *l'insieme di tutte le possibili coppie ordinate x, y , con x elemento dell'insieme A ed y elemento dell'insieme B .*

1.1.2 Successioni matematiche e tuple

Adesso che abbiamo definito formalmente il concetto di insieme e descritto le operazioni fondamentali tra insiemi, vediamo un altro concetto fondamentale: le *successioni* matematiche.

Successione matematica

A differenza degli insiemi, una successione è definita come una lista di elementi disposta secondo un certo ordine. Diamo una definizione più formale.

Def. 1.9 Una *successione matematica* è una sequenza infinita di elementi ordinati in cui la ripetizione di un elemento è ammessa.

Una successione matematica viene descritta con la scrittura formale:

$$a_1, a_2, a_3, \dots, a_n, \dots$$

in cui a_n rappresenta un generico elemento della successione alla posizione n .

Tupla o n-upla

Quando una sequenza è composta da un numero finito di elementi ordinati, ovvero quando è una sequenza *finita*, questa prende il nome di *tupla* (*tuple* in inglese) o *n-upla*, dove n corrisponde alla cardinalità della successione. Una tupla viene spesso indicata dalla notazione:

$$(a_1, a_2, a_3, \dots, a_n),$$

in cui le parentesi tonde vengono utilizzate per differenziarla da un insieme.

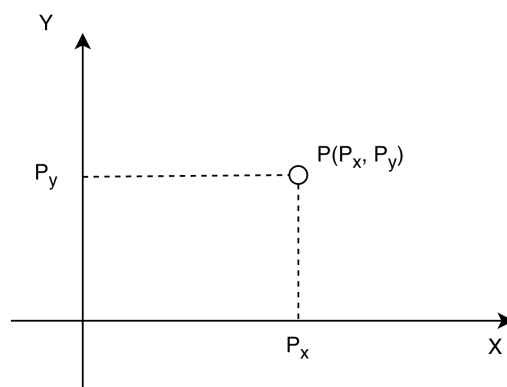


Figura 1.6: Diagramma di un piano cartesiano

Una tupla con due elementi è spesso detta *coppia ordinata*. Un esempio di tupla a due elementi o *coppia ordinata* è la coppia di coordinate del piano cartesiano definita come:

$$(P_x, P_y) = \{x, y \mid x \in X \wedge y \in Y\},$$

in cui X rappresenta l'asse delle ascisse ed Y rappresenta l'asse delle ordinate, come nel diagramma di cui sopra.

Per cui, un'ulteriore analisi dell'operazione di prodotto cartesiano tra insiemi può essere ridefinita come *l'operazione che, dati due insiemi A e B , restituisce un insieme $A \times B$ dato dall'insieme di tutte le possibili tuple o coppie ordinate (x, y) , in cui $x \in A \wedge x \in B$.*

1.1.3 Funzioni e relazioni

Relazione matematica

Per arrivare alla definizione di funzione matematica dobbiamo innanzitutto comprendere cos'è una relazione matematica. Una relazione matematica è definita come un sottoinsieme del prodotto cartesiano tra due insiemi. Diamone, allora, una definizione formale.

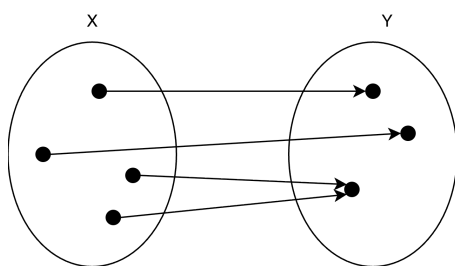
Def. 1.10 Una relazione R è definita come un sottoinsieme del prodotto cartesiano $A \times B$ tra i due insiemi A e B , per cui:

$$R \subseteq A \times B,$$

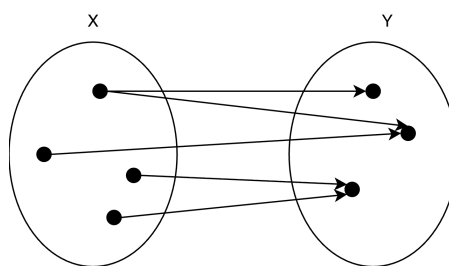
dove A è detto il dominio e B è detto codominio.

Funzione matematica

Più nello, una funzione matematica viene quindi definita come una specifica tipologia di relazione in cui ciascun elemento del dominio viene associato ad uno ed un solo elemento del codominio. Nel diagramma seguente è possibile vedere, sulla sinistra, una rappresentazione di una funzione accettabile, mentre sulla destra un esempio di una relazione che non è una funzione.



(a) Diagramma di una funzione accettabile secondo la definizione



(b) Diagramma di una relazione che non è una funzione

Come abbiamo già definito per una relazione matematica, anche una funzione presenta dominio e codominio. Per descrivere una funzione f che ha dominio D e codominio C si usa la notazione:

$$f : D \rightarrow C$$

Questo tipo di funzione è detta *funzione unaria*.

Inoltre, è possibile visualizzare una funzione con un diagramma astratto come quello a destra.

Una generica funzione f che ha come dominio un insieme di tuple $(a_1, a_2, a_3, \dots, a_n)$ è detta una funzione n -aria, dove n rappresenta il numero di valori in ingresso.

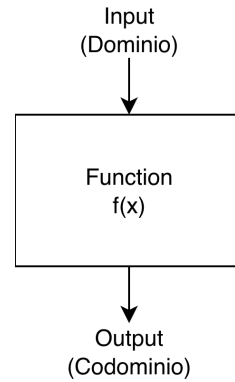


Figura 1.8: Diagramma dell'astrazione di una funzione

Proposizione

Def. 1.11 Quando una funzione f ha codominio $C = \{TRUE, FALSE\}$ è detta *proposizione*.

Il valore assunto da una proposizione è detto *valore di verità* e può essere vero (*true*) oppure falso (*false*): su questa base è costruita l'*algebra booleana*.

Algebra booleana

Introdotta da George Boole (1815-1864) nella sua opera *The Mathematical Analysis of Logic* (1847), l'algebra booleana è un concetto fondamentale del funzionamento degli elaboratori poiché introduce una fondamentale differenza rispetto all'algebra tradizionale: le variabili assumono i valori di verità vero (*true*) e falso (*false*).

Questo permette di utilizzare operatori logici di congiunzione (AND o $\wedge$), disgiunzione inclusiva (OR o $\vee$) ed esclusiva (XOR o $\oplus$) e negazione (NOT o $\neg$) per descrivere le operazioni logiche ed i risultati ad esse associati, come viene fatto di seguito.

A	B	$A \wedge B$	$A \vee B$	$A \oplus B$	$\neg A$
F	F	F	F	F	V
F	V	F	V	V	V
V	F	F	V	V	F
V	V	V	V	F	F

Tabella 1.1: Tavole di verità degli operatori logici AND, OR, XOR e NOT.

1.1.4 Grafi

Un altro concetto fondamentale, che tornerà utile subito dopo questo capitolo introduttivo è il concetto di grafo. Vediamone, quindi, la definizione formale.

Def. 1.12 Un grafo è una coppia ordinata di insiemi, descritta dalla notazione

$$G = (V, E),$$

in cui V è l'insieme di elementi detti nodi o vertici (vertices) ed E è l'insieme di segmenti, descritti dalle coppie non ordinate di vertici $\{v_1, v_2\}$ chiamate archi, lati (edges) o spigoli, in cui v_1 e v_2 sono detti gli estremi dell'arco.

Data la definizione di cui sopra, un grafo può essere rappresentato col diagramma:

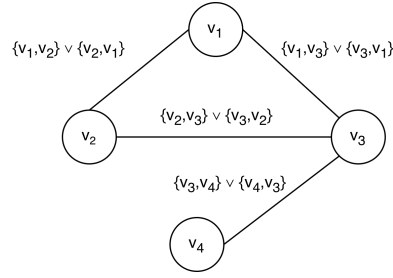


Figura 1.9: Diagramma di un grafo non orientato

Grado di un nodo

Def. 1.13 Il grado di un nodo è dato dalla somma del numero di tutti gli archi di cui esso è uno degli estremi.

Sottografi

Def. 1.14 Un grafo $H = \{V_H, E_H\}$ è detto sottografo di un altro grafo $G = \{V_G, E_G\}$ se tutti gli elementi dell'insieme V_H sono anche elementi dell'insieme V_G e tutti gli elementi dell'insieme E_H sono anche elementi dell'insieme E_G , ovvero se $V_H \subseteq V_G \wedge E_H \subseteq E_G$.

Un sottografo può essere rappresentato dal diagramma seguente (in giallo gli elementi del sottografo):

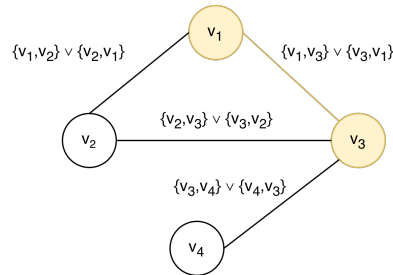


Figura 1.10: Diagramma di un sottografo in un grafo non orientato

Percorsi di un grafo

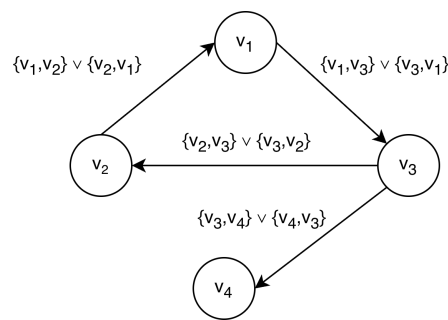
Def. 1.15 *Un percorso in un grafo è una sequenza ordinata di vertici $v_1, v_2, \dots, v_n$ e di archi che li collegano $(v_1, v_2), \dots, (v_{n-1}, v_n)$.*

I vertici v_1 e v_n sono detti gli estremi del percorso. Inoltre, un percorso è detto un *ciclo* se gli estremi del percorso sono dati dallo stesso vertice.

Grafi orientati

Def. 1.16 (h) *Un grafo G è detto orientato quando E_G , cioè l'insieme degli archi del grafo, è formato da coppie ordinate, altrimenti il grafo è detto non orientato.*

Più intuitivamente, un grafo è detto *orientato* se gli archi sono *orientati*, come nel seguente diagramma.



In un grafo orientato, infine, il numero di frecce in uscita da un vertice è detto *grado uscente* di un nodo, mentre il numero di frecce in ingresso da un nodo è detto *grado entrante* di un nodo.

Grafi connessi

Def. 1.17 *Un grafo è detto connesso se qualsiasi coppia di vertici v_k, v_j ha un arco (v_k, v_j) che li connette.*

Figura 1.11: Diagramma di un grafo orientato

Alberi

Def. 1.18 *Un grafo è detto un albero se è connesso e non ha cicli al suo interno.*

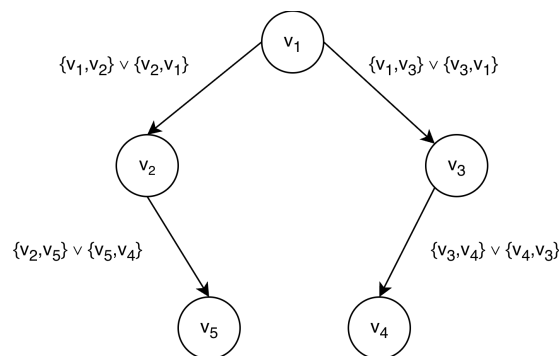


Figura 1.12: Diagramma di un grafo ad albero

1.2 Stringhe

Fondamentale per l'informatica, nel contesto dei linguaggi formali, è anche il concetto di *stringa di caratteri*, ma per essere introdotto, è necessario prima parlare di alfabeto.

Alfabeto

Def. 1.19 *L'alfabeto, indicato dai simboli Σ o Γ , è un insieme finito di elementi detti simboli dell'alfabeto.*

Definizione di stringa Sulla base di un sottoinsieme dell'alfabeto, quindi, si basa il concetto di *stringa di caratteri*, definito come segue.

Def. 1.20 *Una stringa di caratteri S è una sequenza finita di elementi detti simboli in un insieme detto alfabeto.*

Possiamo quindi vedere una stringa S anche come una tupla (o n -upla) di simboli, ovvero

$$S = (s_1, s_2, s_3, \dots, s_n),$$

dove con $s \in \Sigma$ si indica un generico carattere (simbolo) dell'alfabeto.

Lunghezza di una stringa La lunghezza di una generica stringa S , indicata dalla notazione $|S|$ o alternativamente $\#S$, è definita invece come segue.

Def. 1.21 *La lunghezza di una stringa S , indicata dalla notazione $|S|$ o $\#S$, corrisponde al numero di elementi della tupla.*

Quando una stringa S ha lunghezza $|S| = 0$ si parla di *stringa vuota*, che viene indicata con la notazione ε .

1.2.1 Operazioni sulle stringhe

Inversione di una stringa L'inversione di una stringa S , invece, è indicata dal simbolo $S^{\mathcal{R}}$ e corrisponde alla stringa:

$$S^{\mathcal{R}} = (s_n, s_{n-1}, s_{n-2}, \dots, s_1),$$

dove $s \in S$.

Concatenazione di stringhe L'operazione di concatenazione di due stringhe S_1 ed S_2 è definita come segue.

Def. 1.22 *Date due stringhe S_1 ed S_2 , è detta concatenazione delle stringhe l'operazione $S_1 S_2 = (s_{1_1}, s_{1_2}, \dots, s_{1_n}, s_{2_1}, s_{2_2}, \dots, s_{2_n})$.*

Parte I

Concetti di base

2

Rappresentazione dell'informazione

2.1 Sistemi di numerazione

2.2 Conversioni di base

2.3 Aritmetica binaria

2.4 Rappresentazione dei numeri interi

2.5 Numeri in virgola mobile (IEEE 754)

2.6 Rappresentazione di caratteri (ASCII e Unicode)

2.7 Rappresentazione di immagini, audio e video

3

Algebra di Boole e circuiti logici

3.1 Operatori e porte logiche

3.2 Espressioni booleane e tabelle di verità

3.3 Semplificazione tramite mappe di Karnaugh

3.4 Circuiti combinatori

3.5 Circuiti sequenziali

3.6 Dalle porte logiche alla CPU

Parte II

Automi e linguaggi formali

4

Automa a stati finiti

“One day ladies will take their computers for walks in the park and tell each other, “My little computer said such a funny thing this morning”.”

– Alan Turing

Il primo concetto fondamentale del corso di Fondamenti, ignorando i fondamenti già discussi nel capitolo precedente, è quello di definizione formale di automa. Prima di raggiungere tale definizione, però, comprendiamo su quali basi questo viene ideato. La computazione, infatti, nacque prima che la produzione di elaboratori fosse giunta ad uno stadio tale da permetterne uno studio mediante sperimentazione. Lo studio della computazione, infatti, avvenne prima in campo matematico, mediante modelli matematici detti *computational models*. Il primo e più semplice modello matematico utilizzato per descrivere un elaboratore è chiamato *automa a stati finiti*.

Un automa a stati finiti è un modello matematico di computazione particolarmente utile nel contesto di elaboratori con memoria estremamente limitata. Infatti, l'automa a stati finiti consiste in un diagramma degli stati possibili assunti dall'elaboratore e della transizione tra quest'ultimi in base alle condizioni iniziali dell'input. Sulla base di questa descrizione intuitiva, è possibile individuare caratteristiche di un generico problema che può essere modellato mediante un automa a stati finiti:

- **Dinamicità:** la capacità di assumere più stati col trascorrere del tempo;
- **Discretezza:** i valori d'ingresso e gli stati da rappresentare possono essere espressi con valori discreti;
- **Simboli finiti:** il numero di valori d'ingresso e di stati da rappresentare corrispondono ad un valore finito.

Sulla base di questa descrizione, forniamo allora una definizione formale di un'automazione a stati finiti, ovvero:

Def. 4.1 *Un'automazione a stati finiti consiste nella quintupla $(Q, \Sigma, \delta, q_0, F)$, dove con Q si indica l'insieme degli stati possibili dell'elaboratore, con Σ l'alfabeto, δ è detta*

funzione di transizione, q_0 rappresenta lo stato iniziale dell'elaboratore ed F rappresenta l'insieme degli stati in uscita.

4.1 Automa a stati finiti deterministico (DFA)

Un automa a stati finiti, quando deterministico, è caratterizzato da una funzione di transizione $\delta = Q \times \Sigma \rightarrow Q$ in grado di associare alla coppia ordinata $(\Sigma(t), Q(t))$, ovvero di un input $\Sigma(t)$ quando l'elaboratore si trova allo stato attuale $Q(t)$, un nuovo stato $Q(t+1)$. La funzione di transizione può essere espressa più formalmente come:

$$Q(t+1) = \delta(\Sigma(t), Q(t)),$$

dove si indicano:

- con $Q(t+1)$ il nuovo stato assunto dall'automato;
- con $\Sigma(t)$ l'input che genera tale stato;
- con $Q(t)$ lo stato precedente al nuovo.

4.2 Automa a stati finiti non deterministico (NFA)

Un automa a stati finiti, quando non deterministico, corrisponde ad un automa che, dato un input, ha una distribuzione di probabilità di fornire stati in uscita diversi. Un automa a stati finiti non deterministico, quindi, è caratterizzato da una funzione di transizione $\delta = Q \times \Sigma \rightarrow \mathcal{P}(Q)$ che associa alla coppia ordinata $(\Sigma(t), Q(t))$, ovvero ad un input $\Sigma(t)$, quando l'automato si trova allo stato $Q(t)$, una serie di stati in uscita con probabilità diverse, ovvero un sottoinsieme Q_t dell'insieme Q degli stati possibili. La funzione di transizione può quindi essere espressa come:

$$\mathcal{P}(Q(t+1)) = \delta(\Sigma(t), Q(t))$$

4.3 Rappresentazioni di ASF (Automi a Stati Finiti)

4.3.1 Diagramma degli stati

La tradizionale rappresentazione di un automa a stati finiti (ASF) è il grafo orientato di cui abbiamo discusso nello scorso capitolo.

L'esempio di un grafo orientato che rappresenti un automa a stati finiti corrisponde al diagramma visibile a fianco.

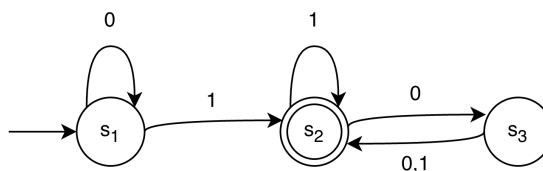


Figura 4.1: Grafo orientato di un ASF

4.3.2 Tabella di transizione di stato

In questo caso, quindi, s_1 corrisponde a q_0 ed $F = \{s_2\}$. Inoltre $Q = \{s_1, s_2, s_3\}$, $\Sigma = \{0, 1\}$ e δ :

$$\delta = \begin{array}{c|cc} & 0 & 1 \\ \hline s_1 & s_1 & s_2 \\ s_2 & s_3 & s_2 \\ s_3 & s_2 & s_2 \end{array}$$

4.4 Linguaggio di un ASF

Come abbiamo già visto, un ASF prende in input una sequenza di simboli appartenenti all'alfabeto Σ . Queste sequenze di simboli appartenenti a Σ sono dette stringhe e l'insieme di quest'ultime è detto linguaggio.

In particolare, è detto linguaggio di una macchina M un linguaggio L composto da stringhe w che la macchina M accetta (per le quali assume stati s nell'insieme dei possibili stati finali F) ed è indicato dalla notazione $L(M) = I$, dove con I si indica un generico insieme di stringhe e si dice che la macchina M riconosce il linguaggio L .

Vediamo quindi più nel dettaglio le caratteristiche di un linguaggio.

5

Linguaggi regolari ed espressioni regolari

Come nell'esempio precedente, alla macchina possono essere forniti input diversi in momenti diversi. Questa sequenza di input è detta *stringa* ed è indicata dalla notazione

$$w = \{w_1, w_2, \dots, w_n\}$$

Ecco che, trattando i linguaggi formali, ci troviamo di fronte alle *stringhe*, altro concetto facente parte dei fondamenti e spiegato nel dettaglio nel primo capitolo del libro. Inoltre, è possibile definire il concetto di *linguaggio* come di un generico insieme L di stringhe w . Proseguendo in questo modo si arriva, quindi, alla definizione di *linguaggio formale*.

Def. 5.1 *Un linguaggio L (cioè un insieme di stringhe) è detto formale se è definito su un alfabeto finito. In particolare, se esiste un automa a stati finiti (non deterministico) che accetta esattamente le stringhe di L , allora L è detto linguaggio regolare.*

Tuttavia, resta da comprendere come decretare se una data stringa w_i sia un input accettabile dall'automa. Per essere un input accettabile, una generica stringa w deve rispettare le seguenti caratteristiche:

- $r_0 = q_0$;
- $\delta(r_i, w_{i+1}) = r_{i+1} \forall i \in [0, n-1]$;
- $r_n \in F$.

Dove con r_i si indica lo stato assunto dall'automa all'input w_i , ovvero: $\delta(r_i, w_i)$. Per cui, secondo le condizioni elencate qui sopra, una generica stringa w è detta input accettabile del generico automa M se il primo input causa il primo stato; se ciascun input che lo segue rispetta la funzione di transizione δ e se l'ultimo input corrisponde ad uno stato elemento dell'insieme F dei possibili stati finali.

Inoltre, un linguaggio è detto *vuoto* se non contiene stringhe ed è indicato dalla notazione $\emptyset$.

5.1 Le operazioni sui linguaggi formali

Sui linguaggi formali, inoltre, è possibile effettuare delle operazioni di unione, concatenazione, intersezione, completamento, quoziente destro, riflesso, mescolamento e di stella di Kleene, definite di seguito.

5.1.1 Unione tra linguaggi

Def. 5.2 *L'unione tra due linguaggi L_1 e L_2 è data da $L_1 \cup L_2 = \{w \mid w \in L_1 \vee w \in L_2\}$, ovvero dall'unione di tutte le stringhe di un linguaggio con tutte le stringhe di un altro.*

Quest'operazione è equivalente all'unione nella teoria degli insiemi, ovvero all'operatore che restituisce un insieme risultante dato da tutti gli elementi del primo insieme, uniti a tutti gli elementi del secondo insieme.

5.1.2 Intersezione di linguaggi

Def. 5.3 *L'operazione di intersezione tra i linguaggi L_1 e L_2 , invece, è data dall'insieme $L_1 \cap L_2 = \{w \mid w \in L_1 \wedge w \in L_2\}$, ovvero dall'insieme composto dalle sole stringhe contenute in entrambi i linguaggi.*

Anche in questo caso, l'operazione di intersezione corrisponde alla medesima operazione nella teoria degli insiemi.

5.1.3 Concatenazione di linguaggi

Def. 5.4 *L'operazione di concatenazione tra due linguaggi L_1 e L_2 corrisponde all'insieme $L_1 \circ L_2 = \{vw \mid v \in L_1 \wedge w \in L_2\}$ di tutte le concatenazioni possibili tra stringhe appartenenti a ciascuno dei due insiemi.*

Per comprendere meglio quest'operazione, prendiamo ad esempio l'alfabeto:

$$\Sigma = \{e, s, m, p, i, o\}$$

In questo caso, due possibili linguaggi costruiti sull'alfabeto potrebbero essere $L_1 = \{seme, mese\}$ e $L_2 = \{peso\}$. L'operazione di concatenazione tra i linguaggi L_1 e L_2 corrisponderebbe, allora all'insieme:

$$L_1 \circ L_2 = \{semepeso, mesepeso\}$$

5.1.4 Complemento di linguaggio

Def. 5.5 *Il complemento di un linguaggio L , indicato dalla notazione $\bar{L}$, è l'insieme di tutte le stringhe w sull'alfabeto Σ che non fanno parte dell'insieme L .*

5.1.5 Quoziente destro di linguaggi

Def. 5.6 Il quoziente destro di un linguaggio L , indicato dalla notazione L_1/L_2 , è l'insieme $L_1/L_2 = \{v \mid vw \in L_1 \wedge w \in L_2\}$.

5.1.6 Stella di Kleene

Def. 5.7 La stella di Kleene di un linguaggio L è l'insieme di tutte le concatenazioni finite di stringhe di L , inclusa la stringa vuota ε :

$$L^* = \bigcup_{n=0}^{\infty} L^n, \quad \text{dove } L^0 = \{\varepsilon\}, \quad L^{n+1} = L^n \circ L.$$

5.1.7 Riflesso di linguaggio

Def. 5.8 L'operatore di riflesso, indicato dalla notazione $L_1^{\mathcal{R}}$ è dato dall'insieme $L_1^{\mathcal{R}} = \{w^{\mathcal{R}} \mid w \in L_1\}$, ovvero dall'insieme di tutte le inversioni di stringhe del linguaggio L .

5.1.8 Mescolamento (Shuffle) tra linguaggi

Def. 5.9 L'operazione di mescolamento (*shuffle*) tra due linguaggi L_1 e L_2 è l'insieme di tutte le stringhe che si possono ottenere interlacciando le lettere di una stringa di L_1 con quelle di una stringa di L_2 , preservando l'ordine originale delle lettere in ciascuna stringa. Formalmente:

$$\{w \in \Sigma^* \mid w = a_1 b_1 \dots a_k b_k \quad \text{dove } k \geq 1 \wedge a_1, \dots, a_k \in L_1 \wedge b_1, \dots, b_k \in L_2\}$$

5.2 Espressioni regolari

A questo punto entra in gioco il concetto di *espressione regolare*. Una stringa R è detta un'espressione regolare se:

- $a \in \Sigma$, ovvero l'espressione regolare corrisponde al linguaggio $\{a\}$;
- $R = \epsilon$, ovvero R è una stringa vuota ϵ e corrisponde al linguaggio $\{\epsilon\}$;
- $R = \emptyset$, ovvero R non contiene stringhe;
- $(R_1 \cup R_2)$, dove R_1 ed R_2 sono a loro volta espressioni regolari;
- $(R_1 \circ R_2)$, dove R_1 ed R_2 sono a loro volta espressioni regolari;
- (R_1^*) , dove R_1 è un'espressione regolare.

5.3 Linguaggi non regolari

Esistono, tuttavia, linguaggi che un generico ASF non è in grado di riconoscere. Questi linguaggi sono detti *linguaggi non regolari*. Per riconoscere un linguaggio non regolare, viene in nostro soccorso il cosiddetto *pumping lemma*. Questo lemma permette di individuare una proprietà caratteristica di ciascun linguaggio regolare. Per cui, se è possibile dimostrare che questa proprietà non vale per un generico linguaggio L , possiamo allora escludere con certezza che questo sia un linguaggio regolare.

5.3.1 Pumping lemma per i linguaggi regolari

Il pumping lemma può essere applicato per determinare se un linguaggio L sia regolare, poiché ciascun linguaggio deve soddisfare tale proprietà per essere considerato regolare, secondo il principio di dimostrazione per contraddizione.

Def. 5.10 *Secondo il pumping lemma, un linguaggio L è detto regolare se $|w| \geq p \forall w \in L$, con p (la pumping length) ≥ 1 , per cui ciascuna stringa w può essere suddivisa come concatenazione delle stringhe xyz , ovvero $w = xyz$, dove $|y| \geq 1$, $|xy| \leq p$ e $xy^n z \in L \forall n \geq 0$.*

Che può essere letta più intuitivamente come:

- y è la sottostringa che può essere "pompatà" ("pumped"), ovvero rimossa o ripetuta un numero qualsiasi n di volte, in modo tale che la stringa risultante sia comunque elemento del linguaggio L e quindi non può essere ϵ , ovvero $|y| \geq 1$;
- xy è la concatenazione delle stringhe x ed y , la cui lunghezza $|xy|$ deve essere al massimo p (la pumping length);
- $xy^n z$ deve essere elemento del linguaggio L a prescindere dal numero n di ripetizioni della sottostringa y .

Per comprendere meglio come il *pumping lemma* sia utile per determinare la regolarità di un linguaggio, analizziamo il seguente esempio.

Dimostrazione. Dato l'automa $M = \{Q, \Sigma, q_0, \delta, F\}$ ed il linguaggio $L = \{a^n b^n : n \geq 0\}$ costruito sull'alfabeto $\Sigma = \{a, b\}$ e l'assunzione che esista un numero p (la *pumping length*) poiché necessario perché il lemma sia vero:

1. Data la stringa $w = a^p b^p$, con $|w| > p$;
2. Allora, per il pumping lemma, deve esistere una suddivisione $w = xyz$ che soddisfa le tre condizioni $|y| \geq 1$ o $y \neq \epsilon$, $|xy| \leq p$ e $xy^n z \in L \forall n \geq 0$;
3. Poiché $|xy| \leq p$, allora y deve necessariamente essere una sottostringa composta da istanze a ;

4. Poiché $|y| \geq 1$, allora y deve contenere almeno un'istanza a ;
5. Ipotizzando di avere $n = 2$, otteniamo una stringa in cui le istanze a devono necessariamente comparire in numero maggiore rispetto alle istanze b , poiché alle istanze a viene applicata una ripetizione mentre alle istanze b non viene applicata alcuna ripetizione;
6. Si dimostra, allora, che il *pumping lemma* non vale per il linguaggio L , per cui L non è un linguaggio regolare, poiché $xy^2z \notin L$.

6

Linguaggi context-free

Altro genere di linguaggi sono i linguaggi cosiddetti *context-free*, nati inizialmente per descrivere il linguaggio umano.

- 6.1 Introduzione alle grammatiche generative
- 6.2 Grammatiche context-free (CFG)
- 6.3 Alberi di derivazione e ambiguità
- 6.4 Forma normale di Chomsky
- 6.5 Automi a pila (PDA)
- 6.6 Equivalenza tra CFG e PDA
- 6.7 Pumping lemma per i linguaggi context-free

7

Macchine di Turing e calcolabilità

- 7.1 Definizione formale della macchina di Turing
- 7.2 Varianti della macchina di Turing
- 7.3 Tesi di Church-Turing
- 7.4 Codifica di macchine di Turing e problemi di decisione
- 7.5 Il problema della fermata (halting problem)
- 7.6 Problemi indecidibili

Parte III

Teoria della computabilità

8

Funzioni ricorsive e modelli di calcolo

8.1 Funzioni primitive ricorsive

8.2 Funzioni μ -ricorsive

8.3 Equivalenza con le macchine di Turing

8.4 La tesi di Church

9

Indecidibilità e riducibilità

9.1 Insiemi ricorsivi e ricorsivamente enumerabili

9.2 Riduzioni many-one e Turing

9.3 Teorema di Rice

9.4 Gerarchia di Chomsky

Parte IV

Teoria della complessità

10

Classi di complessità

- 10.1 Misura della complessità temporale e spaziale
- 10.2 Notazione asintotica O-grande, Ω -grande, Θ -grande
- 10.3 Classi P e NP
- 10.4 Problemi NP-completi e riduzioni polinomiali
- 10.5 Il teorema di Cook-Levin
- 10.6 Oltre NP: classi PSPACE, EXP

11

Trattabilità e intrattabilità

11.1 Algoritmi approssimati

11.2 Classi di complessità nello spazio (L, NL, PSPACE)

11.3 Gerarchie di complessità e problemi aperti (P vs NP)

12

Conclusiooni